



UNIVERSITÀ DI PISA

DEPARTMENT OF COMPUTER SCIENCE

MASTER DEGREE IN COMPUTER SCIENCE

NON-ML PROJECT 5

Pan Zhang
p.zhang@studenti.unipi.it

A.Y. 2025-2026

Contents

1	Introduction	1
1.1	Blockwise KKT condition	1
2	A1: Lagrangian dual decomposition	1
2.1	Equivalent factorized model	1
2.2	Dualizing the linking equation	2
2.3	A target-value deflected subgradient method	3
2.4	Verification of the convergence theorem	5
2.5	Primal feasible upper bounds and stopping rule	6
2.6	A1 pseudocode	8
2.7	Cost per iteration	10
3	A2: general-purpose solver	11
3.1	Direct constrained formulation	11
3.2	Initialization and solution validation	12
3.3	A2 pseudocode	12
3.4	Computational cost	13
4	Implementation and experimental setup	13
4.1	Code organization	13
4.2	Parameters	13
4.3	Instance generation	14
4.4	Reported metrics	15
5	Numerical results	16
5.1	Detailed A1–A2 comparison	16
5.2	Aggregated timing and scaling	17
5.3	Dual bounds and deflection behaviour	19
5.4	Projection diagnostics	20
5.5	Parameter sensitivity	21
5.6	Native PSD input check	23
6	Discussion	23
7	Conclusion	24
A	Reproducibility	25
B	Main formulas implemented in A1	25
C	Native PSD-matrix input	25

1 Introduction

Let $I_1, \dots, I_m$ be pairwise disjoint subsets whose union is $\{1, \dots, n\}$. The feasible set is the Cartesian product of simplices

$$\mathcal{K} = \left\{ x \in \mathbb{R}^n : \sum_{i \in I_k} x_i = 1 \ (k = 1, \dots, m), \ x_i \geq 0 \ (i = 1, \dots, n) \right\}. \quad (1)$$

The primal problem is

$$\min_{x \in \mathcal{K}} f(x) = x^T Q x + q^T x, \quad Q \succeq 0. \quad (\text{P})$$

The set $\mathcal{K}$ is nonempty, compact and convex. Since $Q \succeq 0$, f is convex and continuously differentiable, with

$$\nabla f(x) = 2Qx + q. \quad (2)$$

Therefore, an optimal solution exists. Since the problem is convex, the first-order optimality conditions are also sufficient for global optimality [1].

1.1 Blockwise KKT condition

For a given feasible x , define $g = \nabla f(x)$. On block I_k , the normal cone of the simplex gives the condition

$$g_i \geq \tau_k \quad (i \in I_k), \quad x_i(g_i - \tau_k) = 0 \quad (i \in I_k), \quad (3)$$

for some scalar τ_k . Equivalently, positive variables in the same block must have equal minimum gradient, while zero variables may have larger gradient. The implementation reports the residual

$$\max_k \max_{i \in I_k} |x_i(g_i - \min_{j \in I_k} g_j)| \quad (4)$$

combined with feasibility residuals for block sums and nonnegativity. This is the residual used to verify that A1 and A2 solve the same problem up to numerical tolerance.

2 A1: Lagrangian dual decomposition

2.1 Equivalent factorized model

Because $Q \succeq 0$, write $Q = R^T R$. In the experiments the instances are generated directly through R . When Q is provided directly, the implementation uses a symmetric eigenvalue decomposition $Q = V \text{diag}(\sigma) V^T$ and sets

$$R = \text{diag}(\sqrt{\sigma_i}) V_+^T, \quad (5)$$

where only eigenvalues larger than a numerical tolerance are retained. The objective becomes

$$x^T Q x + q^T x = \|Rx\|_2^2 + q^T x. \quad (6)$$

Introduce a linking variable $y \in \mathbb{R}^r$ and rewrite (P) as

$$\min_{x \in \mathcal{K}, y \in \mathbb{R}^r} \|y\|_2^2 + q^T x \quad \text{s.t.} \quad y = Rx. \quad (\text{P}_R)$$

This formulation is equivalent to (P). The constraint $y = Rx$ is the only coupling between the quadratic term and the simplex variables, and is therefore a natural target for Lagrangian relaxation.

2.2 Dualizing the linking equation

Use a free multiplier $\lambda \in \mathbb{R}^r$ for $Rx - y = 0$. The Lagrangian is

$$L(x, y, \lambda) = \|y\|_2^2 + q^T x + \lambda^T (Rx - y). \quad (7)$$

For fixed λ , minimization over y is explicit:

$$\min_y \|y\|_2^2 - \lambda^T y = -\frac{1}{4}\|\lambda\|_2^2, \quad y(\lambda) = \frac{1}{2}\lambda. \quad (8)$$

The remaining x -subproblem is linear over a product of simplices:

$$\min_{x \in \mathcal{K}} (q + R^T \lambda)^T x. \quad (9)$$

It separates by blocks. If

$$j_k(\lambda) \in \arg \min_{i \in I_k} \{q_i + (R^T \lambda)_i\}, \quad (10)$$

then an exact minimizer is the vertex $x(\lambda)$ defined by $x_{j_k} = 1$ for every block and all other entries equal to zero. Thus the dual function is

$$\psi(\lambda) = -\frac{1}{4}\|\lambda\|_2^2 + \sum_{k=1}^m \min_{i \in I_k} \{q_i + (R^T \lambda)_i\}. \quad (11)$$

This is a finite concave nonsmooth function on $\mathbb{R}^r$. A supergradient is

$$s(\lambda) = Rx(\lambda) - \frac{1}{2}\lambda. \quad (12)$$

Indeed, for any μ , the definition of $x(\lambda)$ and $y(\lambda)$ as minimizers of $L(\cdot, \cdot, \lambda)$ gives

$$\psi(\mu) \leq L(x(\lambda), y(\lambda), \mu) \quad (13)$$

$$= L(x(\lambda), y(\lambda), \lambda) + (\mu - \lambda)^T (Rx(\lambda) - y(\lambda)) \quad (14)$$

$$= \psi(\lambda) + (\mu - \lambda)^T s(\lambda), \quad (15)$$

which is exactly the supergradient inequality for a concave function. The Lagrangian dual problem is

$$\max_{\lambda \in \mathbb{R}^r} \psi(\lambda). \quad (\text{D})$$

Since (PR) is convex, the linking constraints are affine, and the product of simplices has a nonempty relative interior, the relative-interior constraint qualification holds. Strong duality therefore applies [1], and consequently

$$\max(D) = \min(P).$$

2.3 A target-value deflected subgradient method

To cast the dual problem in the minimization framework of Theorem 4.6 of d'Antonio and Frangioni [2, p. 382], we rewrite the dual maximization problem as

$$\min_{\lambda \in \mathbb{R}^r} \Phi(\lambda), \quad \Phi(\lambda) := -\psi(\lambda). \quad (16)$$

Using the expression of the dual function in (11),

$$\Phi(\lambda) = \frac{1}{4} \|\lambda\|^2 - \sum_{k=1}^m \min_{i \in I_k} \{q_i + (R^T \lambda)_i\}. \quad (17)$$

The function Φ is finite, strongly convex and coercive. For a fixed multiplier λ , let $x(\lambda)$ be any exact solution of the blockwise Lagrangian subproblem. Then

$$g(\lambda) = -s(\lambda) = \frac{1}{2} \lambda - R x(\lambda) \in \partial \Phi(\lambda) \quad (18)$$

is an exact subgradient of Φ .

One of the assumptions of Theorem 4.6 is that the search directions are uniformly bounded. Rather than assuming that the multiplier sequence remains in a bounded region, the algorithm explicitly projects every iterate onto a ball guaranteed to contain the unconstrained dual optimum.

Define

$$\rho = 2 \|R\|_F \sqrt{m}, \quad B_\rho = \{\lambda \in \mathbb{R}^r : \|\lambda\| \leq \rho\}. \quad (19)$$

Restricting the dual problem to B_ρ does not change its optimal solution or optimal value. Indeed, since Φ is strongly convex and coercive, it has a unique unconstrained minimizer λ^* .

To characterize the optimality condition, observe that

$$\Phi(\lambda) = \frac{1}{4} \|\lambda\|^2 - \min_{x \in \mathcal{K}} (q + R^T \lambda)^T x.$$

By the subdifferential formula for a pointwise maximum of affine functions,

$$\partial \Phi(\lambda) = \frac{1}{2} \lambda - R \operatorname{conv} \left(\arg \min_{x \in \mathcal{K}} (q + R^T \lambda)^T x \right). \quad (20)$$

Therefore, the optimality condition $0 \in \partial \Phi(\lambda^*)$ implies that there exists

$$\hat{x} \in \operatorname{conv} \left(\arg \min_{x \in \mathcal{K}} (q + R^T \lambda^*)^T x \right) \subseteq \mathcal{K}$$

such that

$$0 = \frac{1}{2} \lambda^* - R \hat{x}.$$

Equivalently,

$$\lambda^* = 2 R \hat{x}. \quad (21)$$

For every $x \in \mathcal{K}$, each block belongs to a simplex and hence

$$\|x_{I_k}\|_2 \leq \|x_{I_k}\|_1 = 1.$$

Consequently,

$$\|x\|^2 = \sum_{k=1}^m \|x_{I_k}\|^2 \leq \sum_{k=1}^m \|x_{I_k}\|_1^2 = m.$$

Using (21),

$$\|\lambda^*\| \leq 2\|R\|_2\|\hat{x}\| \leq 2\|R\|_2\sqrt{m} \leq 2\|R\|_F\sqrt{m} = \rho. \quad (22)$$

Thus $\lambda^* \in B_\rho$, so the restriction to B_ρ leaves the optimum unchanged. The orthogonal projection onto the ball is explicit:

$$\text{proj}_{B_\rho}(z) = \begin{cases} z, & \|z\| \leq \rho, \\ \frac{\rho}{\|z\|}z, & \|z\| > \rho. \end{cases} \quad (23)$$

The method uses the diminishing and square-summable stepsize

$$\nu_k = \frac{\nu_0}{k^p}, \quad \nu_0 > 0, \quad \frac{1}{2} < p \leq 1. \quad (24)$$

This choice satisfies

$$\sum_{k=1}^{\infty} \nu_k = \infty, \quad \sum_{k=1}^{\infty} \nu_k^2 < \infty, \quad (25)$$

which is condition (4.1) in d'Antonio and Frangioni [2, p. 375].

Let

$$\Phi_k = \Phi(\lambda_k), \quad \Phi_{\text{ref}}^k = \min_{1 \leq j \leq k} \Phi_j \quad (26)$$

denote the current value and the best value obtained up to iteration k . Define

$$\delta_k = \frac{\delta_0}{k^\eta}, \quad \delta_0 > 0, \quad \eta > 0, \quad \Phi_{\text{lev}}^k = \Phi_{\text{ref}}^k - \delta_k. \quad (27)$$

Since $\delta_k > 0$ and $\delta_k \rightarrow 0$, threshold condition (4.16) of d'Antonio and Frangioni [2, p. 381] is satisfied.

At the first iteration, set $\alpha_1 = 1$ and $d_1 = g_1$. For $k \geq 2$, define

$$\zeta_k = \frac{\nu_{k-1}\|d_{k-1}\|^2}{\Phi_k - \Phi_{\text{lev}}^k + \nu_{k-1}\|d_{k-1}\|^2}. \quad (28)$$

Because

$$\Phi_k - \Phi_{\text{lev}}^k = \Phi_k - \Phi_{\text{ref}}^k + \delta_k > 0,$$

the denominator is positive and $0 \leq \zeta_k \leq 1$. The implementation selects

$$\alpha_k = \max\{\alpha_{\min}, \zeta_k\}, \quad 0 < \alpha_{\min} \leq 1, \quad (29)$$

and constructs the deflected direction

$$d_k = \alpha_k g_k + (1 - \alpha_k) d_{k-1}. \quad (30)$$

Since both $\alpha_{\min}$ and ζ_k belong to $[0, 1]$,

$$0 \leq \zeta_k \leq \alpha_k \leq 1.$$

Thus (29)–(30) satisfy the target-value deflection rule (4.15) of d’Antonio and Frangioni [2, p. 381].

To make the correspondence with the cited framework explicit, in the notation of d’Antonio and Frangioni [2, eqs. (1.2), (1.5), and (1.6), pp. 358–360], identify

$$X = B_\rho, \quad f = \Phi, \quad x_k = \lambda_k, \quad \bar{g}_k = g_k, \quad v_k = d_{k-1}.$$

Because the previous iteration selects $d_{k-1} = \tilde{d}_{k-1}$, this choice of v_k is one of the alternatives admitted in (1.5). The new deflected direction is

$$\tilde{d}_k = \alpha_k \bar{g}_k + (1 - \alpha_k) v_k,$$

and the implementation selects the admissible unprojected alternative

$$d_k = \tilde{d}_k.$$

Feasibility with respect to B_ρ is enforced by the orthogonal projection in the iterate update. Therefore, the proposed update is covered by Theorem 4.6. Finally, the multiplier update is

$$\lambda_{k+1} = \text{proj}_{B_\rho} (\lambda_k - \nu_k d_k). \quad (31)$$

The primal upper bound is not used in either the stepsize or the deflection rule. It is used only to maintain a feasible primal incumbent and to compute the primal–dual stopping certificate.

2.4 Verification of the convergence theorem

The assumptions of Theorem 4.6 are verified as follows.

Convex model and feasible set. The function Φ is finite and convex. The set B_ρ is nonempty, closed and convex, and it contains the unconstrained minimizer by (22).

Exact oracle. The y -subproblem is solved by (8), and every x -block is solved exactly by (9)–(10). Thus the oracle error in the notation of [2] is

$$\sigma_k = 0 \quad \text{for every } k. \quad (32)$$

Bounded subgradients and directions. Because the update projects onto B_ρ ,

$$\|g_k\| = \left\| \frac{1}{2}\lambda_k - Rx(\lambda_k) \right\| \quad (33)$$

$$\leq \frac{1}{2}\|\lambda_k\| + \|R\|_2\|x(\lambda_k)\| \quad (34)$$

$$\leq \frac{\rho}{2} + \|R\|_2\sqrt{m} \quad (35)$$

$$\leq \rho. \quad (36)$$

Furthermore, $d_1 = g_1$, and (30) is a convex combination because $0 < \alpha_k \leq 1$. By induction,

$$\|d_k\| \leq \rho \quad \text{for every } k. \quad (37)$$

This proves condition (4.10) instead of assuming it. The condition is relevant because the standard projected subgradient recursion contains $\nu_k^2\|d_k\|^2$; bounded directions together with $\sum_k \nu_k^2 < \infty$ make the accumulated squared-step contribution finite.

Stepsize, deflection and threshold conditions. Equations (24)–(25) satisfy condition (4.1); equations (28)–(30) satisfy rule (4.15); and (27) satisfies condition (4.16).

Consider first the infinite sequence obtained when the certified-gap stopping test and the iteration cap are disabled. For that sequence, all assumptions of Theorem 4.6 are satisfied. The theorem gives

$$\Phi_{\text{ref}}^\infty \leq \Phi^* + \sigma^*.$$

Here $\sigma^* = 0$ by (32), while every record value is at least Φ^* . Hence

$$\Phi_{\text{ref}}^k \longrightarrow \Phi^*. \quad (38)$$

Equivalently, define

$$LB_k = \max_{1 \leq j \leq k} \psi(\lambda_j) = -\Phi_{\text{ref}}^k. \quad (39)$$

Using strong duality,

$$LB_k \longrightarrow \psi^* = p^*. \quad (40)$$

The individual values $\psi(\lambda_k)$ need not increase monotonically. The record sequence LB_k is monotone by definition and is the sequence covered by the convergence statement.

The cited theorem establishes asymptotic convergence but does not provide an explicit convergence rate for this variant.

2.5 Primal feasible upper bounds and stopping rule

Every Lagrangian vertex $x(\lambda_k)$ belongs to $\mathcal{K}$, and therefore

$$p^* \leq f(x(\lambda_k)). \quad (41)$$

The implementation also forms the running average

$$\bar{x}_k = \frac{1}{k} \sum_{j=1}^k x(\lambda_j). \quad (42)$$

Since $\mathcal{K}$ is convex, $\bar{x}_k \in \mathcal{K}$, so $f(\bar{x}_k)$ is another valid primal upper bound.

Let

$$\mathcal{C}_k = \{\bar{x}_0\} \cup \{x(\lambda_j), \bar{x}_j : 1 \leq j \leq k\}$$

be the set of feasible primal candidates available after iteration k . The best feasible incumbent and the corresponding upper bound are defined by

$$x_{\text{best},k} \in \arg \min_{x \in \mathcal{C}_k} f(x), \quad UB_k = f(x_{\text{best},k}) = \min \left\{ f(\bar{x}_0), \min_{1 \leq j \leq k} \{f(x(\lambda_j)), f(\bar{x}_j)\} \right\}. \quad (43)$$

Together with the record dual lower bound

$$LB_k = \max_{1 \leq j \leq k} \psi(\lambda_j),$$

weak duality yields

$$LB_k \leq p^* \leq UB_k. \quad (44)$$

Consequently, $UB_k - LB_k$ is a valid primal–dual optimality certificate. The upper bound is used only to maintain a feasible primal incumbent and to evaluate the stopping criterion; it is not used in the stepsize, the deflection rule, or the convergence proof of the dual method.

After the dual loop, a conditional-gradient, or Frank–Wolfe, polishing phase is run from the best feasible incumbent $x_{\text{best},k}$ [3]. For a feasible point x , let

$$h = \nabla f(x) = 2R^T(Rx) + q.$$

The linear minimization oracle is

$$s \in \arg \min_{z \in \mathcal{K}} h^T z, \quad (45)$$

which again separates by blocks and requires one minimum search in each block. Define the conditional-gradient direction

$$d = s - x.$$

Because both x and s belong to $\mathcal{K}$, every update

$$x^+ = x + \gamma d, \quad \gamma \in [0, 1],$$

remains feasible.

The Frank–Wolfe gap at x is

$$\Delta_{\text{FW}}(x) = h^T(x - s) = -h^T d. \quad (46)$$

Since s minimizes $h^T z$ over $\mathcal{K}$ and $x \in \mathcal{K}$, this quantity is nonnegative. Moreover, for every $z \in \mathcal{K}$, convexity gives

$$f(x) - f(z) \leq \nabla f(x)^T(x - z) \leq \Delta_{\text{FW}}(x),$$

so the Frank–Wolfe gap is also a valid first-order optimality certificate for the polishing problem.

Using $Q = R^T R$, let

$$a = d^T Q d = \|Rd\|^2.$$

If $a > 0$, exact line search along the feasible segment gives

$$\gamma^* = \text{proj}_{[0,1]} \left(-\frac{h^T d}{2a} \right) = \text{proj}_{[0,1]} \left(-\frac{\nabla f(x)^T d}{2d^T Q d} \right). \quad (47)$$

If $a = 0$, the objective is affine along the segment. In that case, the algorithm selects $\gamma = 1$ when $h^T d < 0$ and $\gamma = 0$ otherwise. Therefore, every polishing iteration preserves feasibility and cannot increase the primal objective value.

The polishing phase terminates when

$$\Delta_{\text{FW}}(x) \leq \varepsilon_{\text{FW}}, \quad (48)$$

or when the polishing iteration limit T_{pol} is reached.

The dual loop terminates successfully when

$$\frac{UB_k - LB_k}{\max\{1, |UB_k|\}} \leq \varepsilon. \quad (49)$$

If the dual iteration limit T_{max} is reached first, the current valid lower and upper bounds are retained without claiming that the requested tolerance has been attained. A small subgradient norm alone is not used as a stopping criterion, because it does not provide the required primal-dual certificate.

After the polishing phase, the best feasible incumbent and its upper bound are updated whenever the polished point has a smaller objective value. The final relative gap is then recomputed as

$$\text{gap} = \frac{UB - LB}{\max\{1, |UB|\}}.$$

The history $\mathcal{H}$ records the iterations of the dual phase; the final returned values additionally include any improvement obtained during conditional-gradient polishing.

2.6 A1 pseudocode

Algorithm 1 A1: target-value deflected dual subgradient method with conditional-gradient polishing

Input: $R, q, \mathcal{I} = \{I_j\}_{j=1}^m, T_{\text{max}}, \nu_0, p, \delta_0, \eta, \alpha_{\min}, \varepsilon, T_{\text{pol}}, \varepsilon_{\text{FW}}$

Output: Feasible primal point x_{best} , upper bound UB , lower bound LB , relative gap, and dual history $\mathcal{H}$

Phase I: target-value deflected dual method

$\rho \leftarrow 2\|R\|_F \sqrt{m}$

$\lambda_1 \leftarrow 0$

$LB \leftarrow -\infty$

$\Phi_{\text{ref}} \leftarrow +\infty$

$\mathcal{H} \leftarrow []$

Initialize $\bar{x}_0$ by setting $(\bar{x}_0)_i \leftarrow 1/|I_j|$ for every $i \in I_j$

$x_{\text{best}} \leftarrow \bar{x}_0$

$UB \leftarrow f(x_{\text{best}})$

for $k \leftarrow 1$ **to** T_{max} **do**

$c_k \leftarrow q + R^T \lambda_k$

$x_k \leftarrow 0$

for $j \leftarrow 1$ **to** m **do**

$i_j \in \arg \min_{i \in I_j} (c_k)_i \quad \triangleright \text{exact block oracle}$

$(x_k)_{i_j} \leftarrow 1$

end for

$\psi_k \leftarrow -\frac{1}{4} \|\lambda_k\|^2 + \sum_{j=1}^m (c_k)_{i_j}$

$\Phi_k \leftarrow -\psi_k$

$g_k \leftarrow \frac{1}{2} \lambda_k - R x_k$

$LB \leftarrow \max\{LB, \psi_k\}$

$\Phi_{\text{ref}} \leftarrow \min\{\Phi_{\text{ref}}, \Phi_k\}$

$\bar{x}_k \leftarrow \frac{k-1}{k} \bar{x}_{k-1} + \frac{1}{k} x_k$

$F_k \leftarrow f(x_k)$

$\bar{F}_k \leftarrow f(\bar{x}_k)$

if $F_k < UB$ **then**

$UB \leftarrow F_k$

$x_{\text{best}} \leftarrow x_k$

end if

if $\bar{F}_k < UB$ **then**

$UB \leftarrow \bar{F}_k$

$x_{\text{best}} \leftarrow \bar{x}_k$

end if

$\text{gap}_k \leftarrow \frac{UB - LB}{\max\{1, |UB|\}}$

 Append $(k, \psi_k, LB, UB, \text{gap}_k)$ to $\mathcal{H}$

if $\text{gap}_k \leq \varepsilon$ **then**

break

end if

if $k = 1$ **then**

$\alpha_k \leftarrow 1$

$d_k \leftarrow g_k$

else

$\delta_k \leftarrow \frac{\delta_0}{k^\eta}$

$\Phi_{\text{lev}}^k \leftarrow \Phi_{\text{ref}} - \delta_k$

$\zeta_k \leftarrow \frac{\nu_{k-1} \|d_{k-1}\|^2}{\Phi_k - \Phi_{\text{lev}}^k + \nu_{k-1} \|d_{k-1}\|^2}$

$\alpha_k \leftarrow \max\{\alpha_{\text{min}}, \zeta_k\}$

$d_k \leftarrow \alpha_k g_k + (1 - \alpha_k) d_{k-1}$

end if

$\nu_k \leftarrow \frac{\nu_0}{k^p}$

$z_k \leftarrow \lambda_k - \nu_k d_k$

$\lambda_{k+1} \leftarrow \text{proj}_{B_\rho}(z_k)$
end for

Phase II: conditional-gradient polishing

$x \leftarrow x_{\text{best}}$
for $t \leftarrow 0$ **to** $T_{\text{pol}} - 1$ **do**
 $h \leftarrow 2R^T(Rx) + q$
 $s \leftarrow 0$
 for $j \leftarrow 1$ **to** m **do**
 $\ell_j \in \arg \min_{i \in I_j} h_i \quad \triangleright \text{linear minimization oracle}$
 $s_{\ell_j} \leftarrow 1$
 end for
 $d^{\text{FW}} \leftarrow s - x$
 $\Delta_{\text{FW}} \leftarrow h^T(x - s)$
 if $\Delta_{\text{FW}} \leq \varepsilon_{\text{FW}}$ **then**
 break
 end if
 $a \leftarrow \|Rd^{\text{FW}}\|^2$
 if $a > 0$ **then**
 $\gamma \leftarrow \text{proj}_{[0,1]} \left(-\frac{h^T d^{\text{FW}}}{2a} \right)$
 else
 if $h^T d^{\text{FW}} < 0$ **then**
 $\gamma \leftarrow 1$
 else
 $\gamma \leftarrow 0$
 end if
 end if
 $x \leftarrow x + \gamma d^{\text{FW}}$
 $F \leftarrow f(x)$
 if $F < UB$ **then**
 $UB \leftarrow F$
 $x_{\text{best}} \leftarrow x$
 end if
end for
 $\text{gap} \leftarrow \frac{UB - LB}{\max\{1, |UB|\}}$
return $x_{\text{best}}, UB, LB, \text{gap}, \mathcal{H}$

2.7 Cost per iteration

In one iteration of the dual phase, the products $R^T \lambda_k$ and Rx_k dominate the computational work. With sparse storage, each product costs $O(\text{nnz}(R))$; for a dense matrix $R \in \mathbb{R}^{r \times n}$, each product costs $O(rn)$. The blockwise minimum search examines every component once and therefore costs $O(n)$. The target-value scalar calculations cost $O(1)$, while the projection onto B_ρ costs $O(r)$. Consequently, one iteration of the dual phase costs

$$O(\text{nnz}(R) + n + r) \tag{50}$$

in the sparse case and

$$O(rn + n + r)$$

in the dense case.

In one iteration of the conditional-gradient polishing phase, the gradient

$$\nabla f(x) = 2R^T(Rx) + q$$

requires the products Rx and $R^T(Rx)$. Computing the line-search coefficient additionally requires Rd . Therefore, one polishing iteration has the same leading-order sparse complexity,

$$O(\text{nnz}(R) + n),$$

and dense complexity

$$O(rn + n).$$

The memory requirement is dominated by the storage of R and the current primal and dual vectors.

The convergence theorem for the dual method is asymptotic and does not provide a worst-case iteration bound for this variant. The total running time therefore depends on the number of dual iterations performed before the certified-gap test or $T_{\max}$ is reached, together with at most T_{pol} conditional-gradient iterations.

3 A2: general-purpose solver

3.1 Direct constrained formulation

Let $A \in \mathbb{R}^{m \times n}$ be the block-incidence matrix,

$$A_{ki} = \begin{cases} 1, & i \in I_k, \\ 0, & \text{otherwise.} \end{cases}$$

Then A2 solves the original convex QP directly:

$$\min_{x \in \mathbb{R}^n} x^T Q x + q^T x, \quad Ax = \mathbf{1}, \quad x \geq 0. \quad (51)$$

The code supplies the exact objective and gradient

$$f(x) = \|Rx\|^2 + q^T x, \quad \nabla f(x) = 2R^T(Rx) + q,$$

together with the linear equalities and nonnegativity bounds to SciPy's SLSQP implementation. SLSQP is a sequential quadratic programming method originally implemented by Kraft [4], while the software implementation used here is provided by SciPy [5].

3.2 Initialization and solution validation

Since problem (P) is convex, every feasible point satisfying the KKT conditions is globally optimal. The output of A2 is therefore validated through the objective value, the blockwise KKT residual defined in Section 1.1, the maximum block-sum violation, and the maximum nonnegativity violation.

Two A2 configurations are used in the numerical experiments.

- **A2-cold:** SLSQP is initialized independently of A1 at the uniform feasible point

$$(x_{\text{cold}}^0)_i = \frac{1}{|I_k|}, \quad i \in I_k, \quad k = 1, \dots, m.$$

- **A2-warm:** SLSQP is initialized at the polished feasible point returned by A1:

$$x_{\text{warm}}^0 = x_{\text{A1}}.$$

Both configurations use exactly the same objective function, exact gradient, equality constraints, bound constraints, and SLSQP options. In particular, both use

$$\text{ftol} = 10^{-10} \quad \text{and} \quad T_{\text{A2}} = 800.$$

Hence, the initial point is the only algorithmic difference between the two A2 runs.

A2-cold measures the independent performance of the general-purpose solver. A2-warm measures the benefit of refining a solution already produced by A1. The wall-clock time of A2-warm alone contains only the refinement phase and excludes the cost of producing the initial point. For this reason, the report also considers the end-to-end pipeline time

$$t_{\text{pipeline}} = t_{\text{A1}} + t_{\text{A2,warm}}. \tag{52}$$

The A2-warm time must therefore not be interpreted as the independent running time of A2 or compared directly with the total running time of A1 without also reporting (52).

3.3 A2 pseudocode

Algorithm 2 A2: cold-start and A1 warm-start constrained solves

Input: $R, q, \{I^j\}_{j=1}^m$, polished A1 solution x^{A1} , maximum iterations $T_{\text{max}}^{\text{A2}}$, tolerance ε_{A2}

Output: A2-cold result, A2-warm result, pipeline time, solver statuses, and residuals

Build $A \in \mathbb{R}^{m \times n}$ with $A_{ji} = 1$ for $i \in I^j$

Construct the uniform feasible point x^{unif}

for $j = 1, \dots, m$ **do**

for $i \in I^j$ **do**

$(x^{\text{unif}})_i \leftarrow 1/|I^j|$

end for

end for

Define $f(x) = \|Rx\|^2 + q^T x$

Define $\nabla f(x) = 2R^T(Rx) + q$

Call SLSQP from x^{unif} on $\min f(x)$ subject to $Ax = \mathbf{1}$ and $x \geq 0$
 Store the returned point $x^{A2,\text{cold}}$, objective value, status, iterations, timing, and residuals
 Call SLSQP from x^{A1} on $\min f(x)$ subject to $Ax = \mathbf{1}$ and $x \geq 0$
 Store the returned point $x^{A2,\text{warm}}$, objective value, status, iterations, timing, and residuals
 Compute $t_{\text{pipeline}} \leftarrow t_{A1} + t_{A2,\text{warm}}$
return $x^{A2,\text{cold}}$, $x^{A2,\text{warm}}$, their objective values, statuses, iterations, timings, residuals, and t_{pipeline}

3.4 Computational cost

One objective evaluation requires Rx and costs $O(\text{nnz}(R) + n)$ in sparse arithmetic or $O(rn + n)$ for dense R . One gradient evaluation additionally computes $R^T(Rx)$ and has the same leading order. The block constraints cost $O(n)$ to evaluate. The total A2 cost also contains the dense linear-algebra work internal to SLSQP, which depends on the active set and cannot be summarized by a single problem-independent iteration bound. The numerical results report the number of SLSQP iterations, objective-function and gradient evaluations, and wall-clock time.

4 Implementation and experimental setup

4.1 Code organization

The implementation is written in Python. NumPy is used for numerical linear algebra, while SciPy is used only by A2. The repository is organized as follows:

```

code/
  qpsimplex/
    problem.py model, PSD factorization, generation, save/load
    dual_deflected.py A1 target-value deflected dual method
    primal_recovery.py optional conditional-gradient polishing
    a2_scipy.py A2 general-purpose SLSQP solver
    metrics.py objective, feasibility and KKT residuals
    experiments.py reproducible comparison grid
  scripts/
    run_one.py solve one generated or saved instance
    run_experiments.py reproduce the A1/A2 comparison
    make_report_assets.py regenerate tables and figures
    run_self_check.py algebraic, theorem-rule and smoke checks
  results/ generated CSV files and saved instances
  report_assets/ generated LaTeX tables and PDF figures
report.tex this report

```

4.2 Parameters

The main A1 configuration uses

$$\nu_0 = 0.5, \quad p = 0.75, \quad \delta_0 = 1, \quad \eta = 0.5, \quad \alpha_{\min} = 0.1, \quad T_{\max} = 5000.$$

These values satisfy the theoretical conditions stated in Section 2. Their finite-iteration effect is examined in the parameter sensitivity experiment.

For A2, both the cold-start and warm-start runs use SciPy SLSQP with the same settings:

$$\text{ftol} = 10^{-10}, \quad T_{\text{A2}} = 800.$$

The cold-start run uses the uniform feasible point

$$(x_{\text{cold}}^0)_i = \frac{1}{|I_k|}, \quad i \in I_k,$$

whereas the warm-start run uses

$$x_{\text{warm}}^0 = x_{\text{A1}}.$$

All remaining solver options, objective and gradient functions, constraints, and bounds are identical. Consequently, any difference between the two A2 runs is attributable to the initial point and to ordinary runtime variability.

The reported wall-clock times are obtained from one execution of each solver configuration on each generated instance. Therefore, the timing results should be interpreted as implementation-level measurements on the current hardware rather than as precise performance estimates. This limitation is particularly relevant for the smallest instances, for which the A2 execution time is only a few milliseconds.

The reported A1 time includes both the target-value deflected dual phase and the subsequent conditional-gradient polishing phase. The A2-cold and A2-warm times contain only the corresponding SLSQP calls. The end-to-end warm-start pipeline time is therefore computed as

$$t_{\text{pipeline}} = t_{\text{A1}} + t_{\text{A2,warm}}.$$

4.3 Instance generation

The numerical instances are generated through a factor $R \in \mathbb{R}^{r \times n}$, so that

$$Q = R^T R$$

is positive semidefinite by construction. Before applying the density mask, the entries are sampled independently according to

$$R_{ji} \sim \mathcal{N}(0, 1), \quad q_i \sim \mathcal{N}(0, 1).$$

When the prescribed density is smaller than one, a random binary mask is applied to the entries of R . The resulting matrix is then scaled by

$$\frac{1}{\sqrt{r\rho_R}},$$

where ρ_R denotes the target density. This scaling keeps the quadratic term at a comparable numerical magnitude across the tested families.

The four instance families used in the reported experiments are:

- **small dense:** $n = 50$, $m = 5$, block size 10, $r = 20$, density 1.00;
- **medium dense:** $n = 100$, $m = 10$, block size 10, $r = 35$, density 1.00;
- **medium sparse:** $n = 150$, $m = 15$, block size 10, $r = 45$, density 0.20;
- **large sparse:** $n = 160$, $m = 20$, block size 8, $r = 50$, density 0.12.

Two independent random seeds are used for every family. The generated instances are stored in `code/results/data/`, so that all reported comparisons can be repeated on exactly the same numerical inputs.

The labels “medium sparse” and “large sparse” refer to the zero pattern and target density of R . For portability, the current implementation still stores and processes the masked matrices as dense NumPy arrays. Therefore, the reported wall-clock times are based on dense linear algebra. The $O(\text{nnz}(R))$ complexity discussed in Sections 2.7 and 3.4 refers to an implementation using an actual sparse matrix representation.

4.4 Reported metrics

For every test instance, the experiments report the following quantities:

- the best feasible objective value produced by A1 after conditional-gradient polishing;
- the record dual lower bound of A1 and the relative primal–dual certificate

$$\text{gap}_{A1} = \frac{UB - LB}{\max\{1, |UB|\}};$$

- the objective values, iteration counts, solver statuses, elapsed times, feasibility residuals, and blockwise KKT residuals of A2-cold and A2-warm;
- the relative objective discrepancies

$$\Delta_{\text{cold}} = \frac{|f_{A1} - f_{A2,\text{cold}}|}{\max\{1, |f_{A2,\text{cold}}|\}},$$

and

$$\Delta_{\text{warm}} = \frac{|f_{A1} - f_{A2,\text{warm}}|}{\max\{1, |f_{A2,\text{warm}}|\}};$$

- the warm-start speedup

$$S_{\text{warm}} = \frac{t_{A2,\text{cold}}}{t_{A2,\text{warm}}}; \tag{53}$$

- the end-to-end A1–A2 pipeline time

$$t_{\text{pipeline}} = t_{A1} + t_{A2,\text{warm}};$$

- the pipeline-to-cold ratio

$$R_{\text{pipeline}} = \frac{t_{\text{pipeline}}}{t_{\text{A2,cold}}}. \quad (54)$$

A value $S_{\text{warm}} > 1$ means that warm-starting decreases the internal A2 refinement time. A value $R_{\text{pipeline}} < 1$ means that the complete A1–A2 pipeline is faster than independently running A2 from its default initial point. The quantities answer different questions and are therefore reported separately.

5 Numerical results

5.1 Detailed A1–A2 comparison

Table 1 reports the results for A1, independently run cold-start A2, and warm-start A2. The column labelled “pipeline” reports the end-to-end time

$$t_{\text{pipeline}} = t_{\text{A1}} + t_{\text{A2,warm}},$$

whereas the warm-start speedup is defined as

$$S_{\text{warm}} = \frac{t_{\text{A2,cold}}}{t_{\text{A2,warm}}}.$$

Table 1: Detailed comparison of A1, cold-start A2, and warm-start A2. The pipeline time is $t_{\text{A1}} + t_{\text{A2,warm}}$, while the warm-start speedup is $t_{\text{A2,cold}}/t_{\text{A2,warm}}$.

Instance	n	A1 obj.	A2-c obj.	A2-w obj.	cert. gap	A1 it.	A1 s	A2-c it.	A2-c s	A2-w it.	A2-w s	pipeline s	warm speedup	max KKT
small_dense_seed0	50	-5.0942	-5.0942	-5.0942	1.53e-05	5000	0.11	12	0.00	4	0.00	0.11	2.96	5.54e-06
small_dense_seed1	50	-8.0868	-8.0868	-8.0868	8.05e-06	862	0.03	10	0.00	5	0.00	0.03	2.04	6.96e-06
medium_dense_seed0	100	-10.2023	-10.2023	-10.2023	2.38e-05	5000	0.12	17	0.02	8	0.01	0.13	2.54	3.53e-05
medium_dense_seed1	100	-13.3512	-13.3512	-13.3512	1.75e-05	5000	0.17	19	0.02	7	0.02	0.18	1.12	3.88e-05
medium_sparse_seed0	150	-19.6146	-19.6147	-19.6147	1.92e-05	5000	0.21	21	0.07	8	0.02	0.23	2.81	6.26e-05
medium_sparse_seed1	150	-15.9528	-15.9530	-15.9530	3.74e-05	5000	0.22	22	0.07	9	0.03	0.25	2.42	9.65e-05
large_sparse_seed0	160	-17.2342	-17.2345	-17.2345	4.72e-05	5000	0.26	31	0.10	13	0.05	0.31	2.23	1.91e-04
large_sparse_seed1	160	-18.3644	-18.3646	-18.3646	4.51e-05	5000	0.21	27	0.10	13	0.05	0.26	2.02	9.18e-05

Across the tested instances, A2-cold and A2-warm reach essentially the same objective values. Hence, the initialization mainly affects the computational effort rather than the final solution reached.

The comparison between A1 and A2-cold is the appropriate independent algorithm comparison, because neither method uses the output of the other. In contrast, the comparison between A2-cold and A2-warm isolates the effect of the starting point. The pipeline time measures the actual end-to-end cost when A1 is run in order to initialize A2.

The largest absolute relative objective discrepancy between A1 and A2-cold is

$$2.21 \times 10^{-5},$$

while the largest final A1 relative primal–dual certificate is

$$4.73 \times 10^{-5}.$$

The A1 points are feasible to machine precision, and their largest reported blockwise KKT residual is

$$1.91 \times 10^{-4}.$$

Only one A1 run reaches the requested 10^{-5} certificate before the iteration limit. The remaining runs correctly report the `max_iter` status together with valid lower and upper bounds, rather than claiming convergence to the requested tolerance.

5.2 Aggregated timing and scaling

Table 2 summarizes the average performance by instance family. The reported quantities distinguish the independent comparison between A1 and A2-cold, the effect of warm-starting on the SLSQP phase, and the complete cost of the A1–A2 warm-start pipeline.

Table 2: Average computational performance by instance family. A2-cold denotes an independent SLSQP run from the uniform feasible point. A2-warm denotes only the SLSQP refinement started from the polished A1 solution. The pipeline time is $t_{A1} + t_{A2,warm}$.

Family	n	A1 s	A2-c s	A2-w s	pipeline s	cold/A1	pipeline/cold	warm speedup	max rel. diff.
small_dense	50	0.07	0.00	0.00	0.07	0.0	31.04	2.50	3.34e-09
medium_dense	100	0.15	0.02	0.01	0.16	0.1	8.12	1.83	4.72e-06
medium_sparse	150	0.22	0.07	0.03	0.24	0.3	3.63	2.61	1.03e-05
large_sparse	160	0.24	0.10	0.05	0.28	0.4	2.77	2.12	2.20e-05

Independent comparison. A1 must be compared with A2-cold when the two methods are evaluated as independent solvers. The column labelled “cold/A1” reports

$$\frac{t_{A2,cold}}{t_{A1}}.$$

On all tested instance families, this ratio is smaller than one. Therefore, under the current implementation, hardware, stopping rules, and experiment grid, independently run A2-cold has a lower measured wall-clock time than A1.

This timing difference is consistent with the implementations. A1 performs up to 5000 structured dual iterations and subsequently runs a conditional-gradient polishing phase, whereas SLSQP terminates after a substantially smaller number of iterations on the tested instances. The reported A1 time also includes the polishing phase.

Effect of warm-starting. The warm-start speedup is defined as

$$S_{warm} = \frac{t_{A2,cold}}{t_{A2,warm}}.$$

The reported values are larger than one for all tested families. Hence, starting SLSQP from the polished A1 point reduces the internal A2 refinement time. The warm start is therefore computationally useful when the A1 solution is already available.

End-to-end pipeline cost. The A2-warm time alone excludes the cost of producing the A1 initial point. The complete cost of using A1 specifically to initialize A2 is

$$t_{\text{pipeline}} = t_{\text{A1}} + t_{\text{A2,warm}}.$$

The column labelled “pipeline/cold” reports

$$\frac{t_{\text{pipeline}}}{t_{\text{A2,cold}}}.$$

This ratio is larger than one for every tested family. Consequently, although warm-starting reduces the SLSQP refinement time, the complete A1–A2 pipeline is slower than independently running A2-cold on the current experiment grid.

The cold-start experiment therefore changes the interpretation of the original warm-start-only comparison. Under the reported experimental conditions, A2-cold has the lowest measured wall-clock time. A1’s distinguishing advantage is not speed on these instances, but the valid dual lower bound and primal–dual optimality certificate that it provides.

Because the smallest A2 measurements are only a few milliseconds and each configuration is timed once, the numerical ratios should not be interpreted as general performance or complexity claims.

Figure 1 shows the absolute relative objective discrepancy between A1 and independently run A2-cold. Figure 2 compares the four relevant timing quantities.

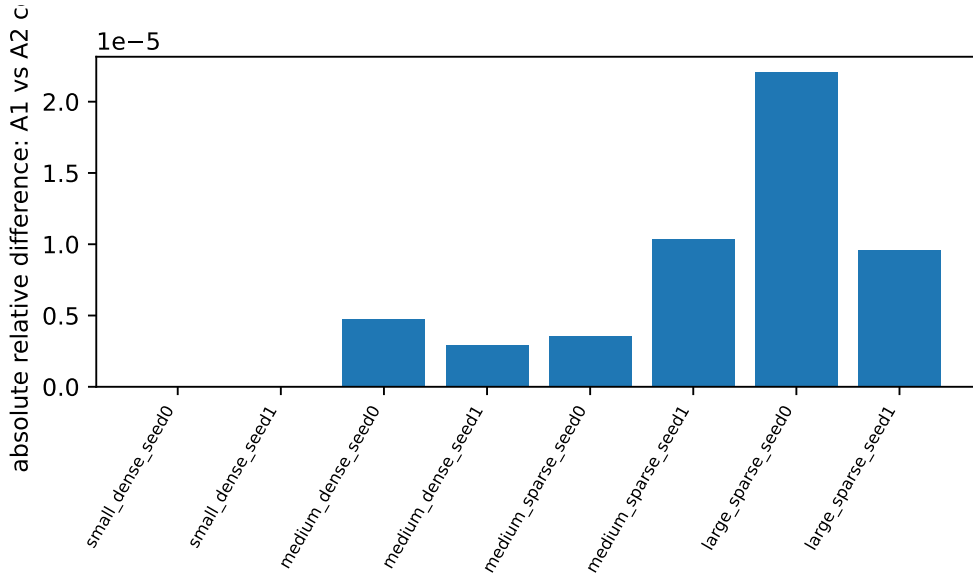


Figure 1: Absolute relative objective difference between A1 and independently run cold-start A2.

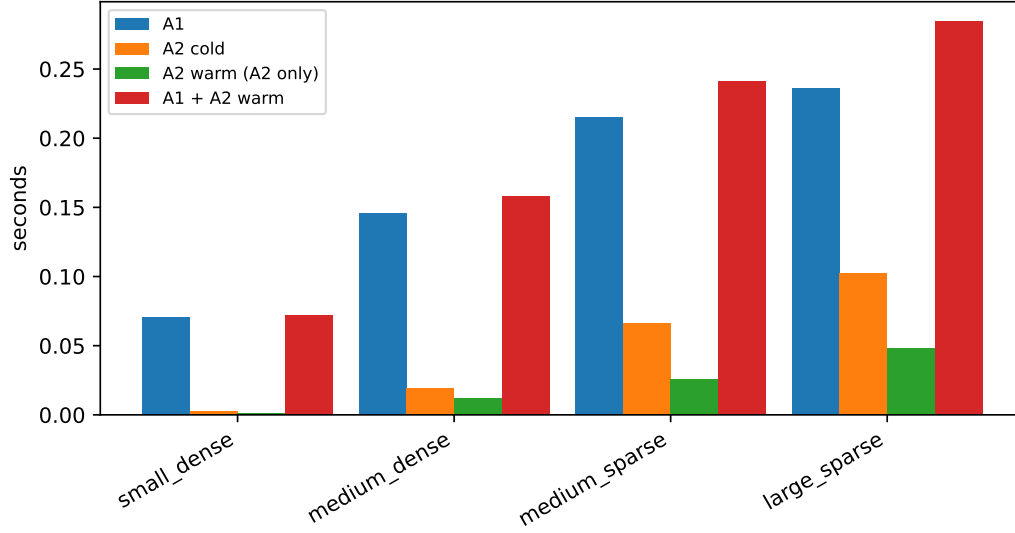


Figure 2: Wall-clock times for A1, independently run A2-cold, the A2-only warm-start refinement, and the end-to-end A1–A2 pipeline.

5.3 Dual bounds and deflection behaviour

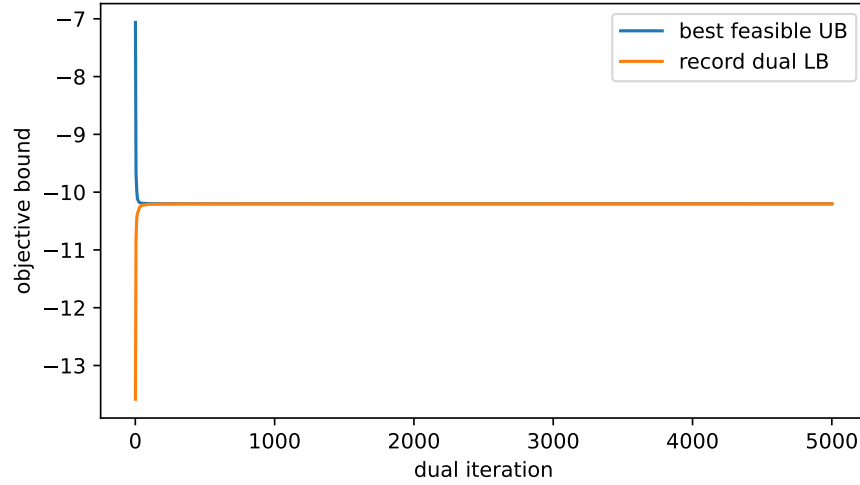


Figure 3: Record dual lower bound and best feasible upper bound on a representative medium dense instance.

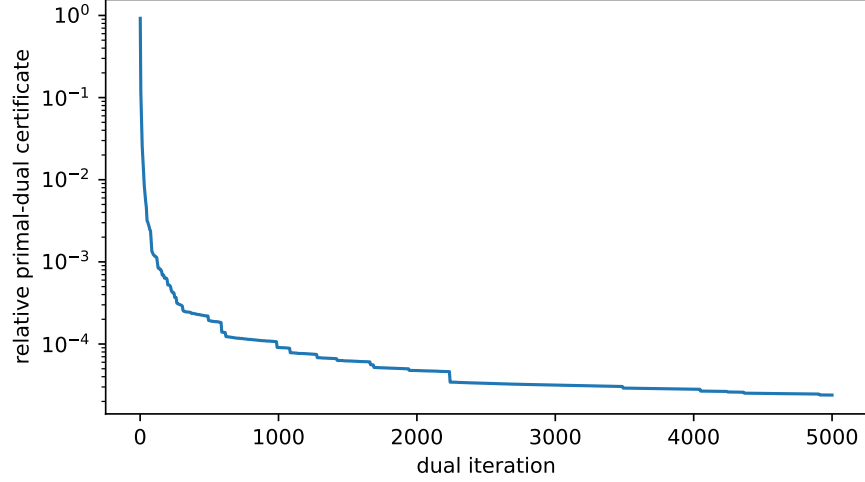


Figure 4: Relative primal–dual certificate for the same instance.

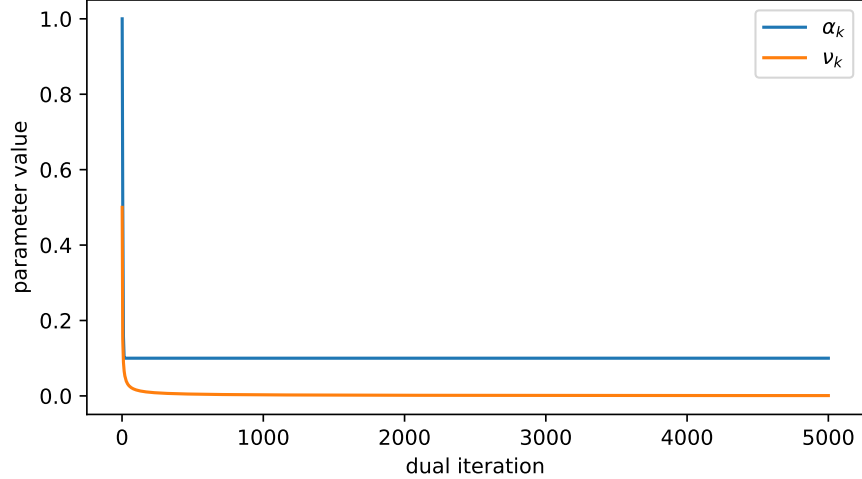


Figure 5: Diminishing stepsize ν_k and target-value deflection coefficient α_k .

The theorem concerns the record dual values, not monotonicity of each raw dual value. The plots therefore display LB_k . The feasible upper bound is produced by a separate primal-recovery mechanism and may improve at a different speed.

5.4 Projection diagnostics

The projection onto B_ρ is included to guarantee that the multiplier sequence remains in a bounded set containing a dual optimum. Consequently, the subgradients and deflected directions are uniformly bounded, as required by the convergence theorem.

The recorded multiplier norms remain below the radius ρ on the tested instances. This confirms that every stored iterate satisfies the imposed ball constraint. However, the current

implementation does not separately record the norm of every unprojected trial point or the number of iterations at which the projection changes the trial point. Therefore, no numerical claim is made here about how often the projection is active.

Regardless of its observed numerical activity, the projection remains essential to the theoretical analysis because it provides an explicit bounded feasible set for the dual iteration.

5.5 Parameter sensitivity

The parameter-sensitivity experiment is performed on one fixed medium dense instance. Only the A1 algorithmic parameters are varied, while the problem data, iteration limits, polishing procedure, and A2 reference solution are kept unchanged.

In this experiment, independently run A2-cold is used as the reference solution. No initial point is supplied to SLSQP, so A2 uses its default uniform feasible point. Therefore, the reference solution is independent of the output of A1 and does not benefit from an A1 warm start.

The tested A1 configurations vary the initial stepsize ν_0 , the stepsize exponent p , and the minimum deflection coefficient $\alpha_{\min}$. All tested configurations satisfy the convergence restrictions

$$\nu_0 > 0, \quad \frac{1}{2} < p \leq 1, \quad 0 < \alpha_{\min} \leq 1.$$

Table 3 reports the relative objective difference with respect to A2-cold, the final A1 primal-dual certificate, and the blockwise KKT residual.

Table 3: Sensitivity on one fixed medium dense instance. Every tested setting satisfies the convergence restrictions. The A2 reference is independently run A2-cold from the uniform feasible point.

Setting	ν_0	p	$\alpha_{\min}$	rel. diff. vs A2	cert. gap	KKT
nu0=0.2	0.2	0.75	0.10	5.74e-06	2.01e-04	5.71e-05
nu0=0.5	0.5	0.75	0.10	6.82e-06	2.89e-05	5.11e-05
nu0=1.0	1.0	0.75	0.10	1.79e-05	5.70e-05	8.40e-05
p=0.60	0.5	0.60	0.10	6.97e-06	7.44e-05	6.09e-05
p=0.90	0.5	0.90	0.10	6.82e-06	6.31e-05	5.07e-05
alpha_min=0.3	0.5	0.75	0.30	6.77e-06	3.58e-05	5.29e-05

In Table 3, the column labelled “rel. diff. vs A2” refers specifically to the relative objective difference with respect to A2-cold:

$$\Delta_{\text{cold}} = \frac{f_{A1} - f_{A2,\text{cold}}}{\max\{1, |f_{A2,\text{cold}}|\}}.$$

The label is retained because it is generated automatically by the current report-asset script; throughout this section, “A2” in that column means A2-cold.

All tested configurations produce objective values close to the A2-cold reference solution. The absolute relative objective differences remain of order 10^{-5} or smaller. The corresponding KKT residuals are also small, indicating that the polished A1 points are close to satisfying the first-order optimality conditions.

The results show that the algorithmic parameters mainly affect the finite-iteration primal–dual certificate rather than the final objective value. In particular, the tested value

$$\nu_0 = 0.5$$

produces the smallest certificate among the three tested initial stepsizes. The smaller value $\nu_0 = 0.2$ gives slower progress within the fixed iteration budget, while the larger value $\nu_0 = 1.0$ leads to a somewhat larger final objective discrepancy and KKT residual.

The tested values of the stepsize exponent p also produce similar final objective values, but different primal–dual certificates. This is consistent with the role of p in controlling the rate at which the stepsize

$$\nu_k = \frac{\nu_0}{k^p}$$

decreases. A smaller value of p preserves larger steps for longer, whereas a larger value decreases the steps more rapidly.

Increasing $\alpha_{\min}$ from 0.1 to 0.3 changes the minimum weight assigned to the current subgradient in the deflected direction. On this test instance, the change has only a limited effect on the objective value, certificate, and KKT residual.

Figure 6 shows the effect of the initial stepsize ν_0 on the final relative primal–dual certificate.

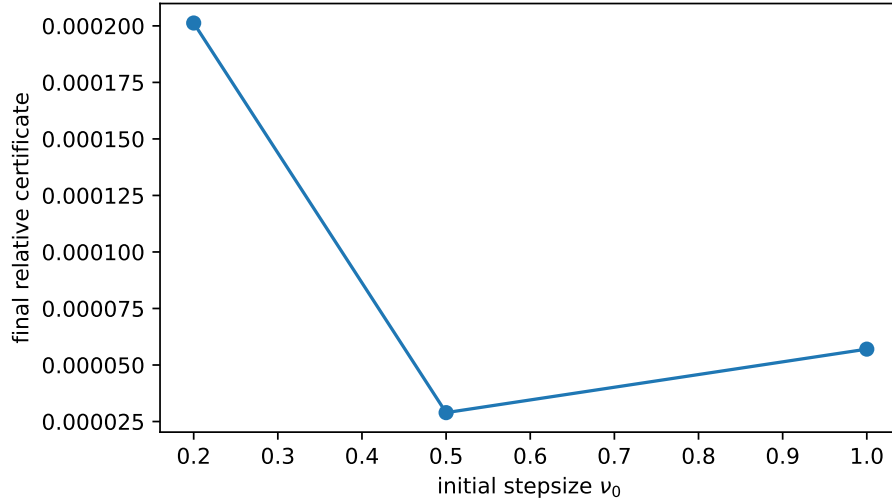


Figure 6: Effect of the initial stepsize ν_0 on the final finite-iteration primal–dual certificate for one fixed medium dense instance.

The sensitivity experiment is not intended to change the mathematical model or to identify universally optimal parameters. It evaluates only the finite-iteration behaviour of A1 on one

fixed instance. The baseline values

$$\nu_0 = 0.5, \quad p = 0.75, \quad \alpha_{\min} = 0.1$$

are retained because they provide a comparatively small final certificate and stable numerical behaviour on the tested instance, not because they are claimed to be theoretically optimal for all problems.

5.6 Native PSD input check

The main experiments generate the quadratic matrix through a factor R , so that $Q = R^T R$ is positive semidefinite by construction. Nevertheless, the implementation also accepts a symmetric positive semidefinite matrix Q directly. In this case, `SimplexQP.from_Q` reconstructs a factor R using the symmetric eigendecomposition described in (5), retaining only the eigenvalues larger than the prescribed numerical tolerance.

A dedicated PSD-input validation is implemented in `scripts/make_report_assets.py`. It constructs an instance from a factor R , reconstructs a second representation from the corresponding matrix $Q = R^T R$, and compares

$$x^T Q x + q^T x \quad \text{and} \quad \|R x\|_2^2 + q^T x$$

on twenty randomly generated feasible points. The maximum absolute objective discrepancy is written to `code/results/psd_factorization_check.txt`. The observed discrepancy is at the level of floating-point roundoff, confirming that the native- Q representation and the factorized representation define the same quadratic objective up to numerical precision.

Separately, `scripts/run_self_check.py` verifies the same factorization on the uniform feasible point before performing the A1 and A2 consistency checks.

6 Discussion

Dualizing the linking equality $y = R x$ makes every A1 Lagrangian relaxation exactly solvable. For a fixed multiplier, the y -subproblem has a closed-form solution, while the x -subproblem reduces to one minimum reduced-cost search on each simplex block. Thus, A1 satisfies the no-solver requirement. Directly dualizing the simplex equalities would instead leave a nonnegative quadratic optimization problem inside the oracle.

The theoretical guarantee concerns the record dual lower-bound sequence. Under the assumptions verified in Section 2, this sequence converges asymptotically to the primal optimum. The result does not imply monotonicity of the individual dual values or provide a worst-case convergence rate for the implemented deflected subgradient variant.

Every reported A1 upper bound is obtained from an explicitly feasible point. Together with the record dual lower bound, this gives the valid certificate

$$\frac{UB - LB}{\max\{1, |UB|\}}.$$

Because the exact Lagrangian solutions are simplex vertices, running averages and conditional-gradient polishing are used to obtain better primal points when the optimum is not a vertex.

In the experiments, most A1 runs reach the iteration limit before satisfying the requested certificate, but they still return valid lower and upper bounds without claiming convergence.

The A2 experiments separate two different questions. A2-cold starts from the uniform feasible point and is the appropriate reference for comparing A1 and A2 as independent solvers. A2-warm starts from the polished A1 solution and measures the benefit of initialization. On the tested instances, warm-starting reduces the SLSQP refinement time, but the complete cost

$$t_{A1} + t_{A2, \text{warm}}$$

is larger than the A2-cold time on every tested family. Hence, running A1 solely to initialize A2 is not computationally advantageous on this experiment grid.

The methods nevertheless provide different information. A2-cold has the lowest measured wall-clock time, whereas A1 additionally returns a valid dual lower bound and a primal–dual certificate. A2-warm remains useful when an A1 solution has already been computed for certification or analysis and only an additional primal refinement is desired.

These timing conclusions are specific to the current implementation, hardware, solver settings, stopping rules, and test instances. Since each configuration is timed once and some A2 runs last only a few milliseconds, the reported ratios are implementation-level observations rather than general performance or complexity claims.

7 Conclusion

A1 is a solver-free Lagrangian dual decomposition for convex quadratic programmes over a product of simplices. Its oracle is solved exactly, the target-value deflected subgradient update satisfies the assumptions of the cited convergence theorem, and projection onto an explicitly constructed ball guarantees bounded multipliers and directions. A1 returns a feasible primal point, a valid dual lower bound, and a primal–dual optimality certificate.

A2 applies SciPy’s SLSQP routine directly to the original constrained problem. To evaluate the effect of initialization fairly, it is run both from the default uniform feasible point (A2-cold) and from the polished A1 solution (A2-warm). The two configurations reach essentially the same objective values, showing that initialization mainly affects computational effort.

On the tested instances, A2-cold has the lowest measured wall-clock time. Warm-starting reduces the A2 refinement time, but the complete pipeline

$$t_{A1} + t_{A2, \text{warm}}$$

is slower than A2-cold on every tested family. Therefore, A1 should not be run solely to initialize A2. Its main advantage is instead the certified primal–dual information that SLSQP does not directly provide.

The final comparison therefore distinguishes three roles: A2-cold is the faster standalone implementation on the tested instances, A1 provides certification without a general-purpose solver, and A2-warm offers an optional refinement when an A1 solution is already available. These conclusions are empirical and specific to the reported implementation and experimental setup.

A Reproducibility

The Python commands are executed from the `code/` directory, while `report.tex` is compiled from the project root. A Conda environment can be created with:

```
conda create -n nonml5 python=3.11 -y
conda activate nonml5
conda install -c conda-forge numpy scipy pandas matplotlib -y
```

From the project root, reproduce the numerical results with:

```
cd code
python scripts/run_self_check.py
python scripts/run_experiments.py
python scripts/make_report_assets.py
cd ..
```

The scripts regenerate the saved instances and CSV files under `code/results/`, and the LaTeX tables and PDF figures under `code/report_assets/`.

B Main formulas implemented in A1

$$\begin{aligned}\psi(\lambda) &= -\frac{1}{4}\|\lambda\|^2 + \sum_k \min_{i \in I_k} \{q_i + (R^T \lambda)_i\}, \\ g(\lambda) &= \frac{1}{2}\lambda - Rx(\lambda), \\ \nu_k &= \nu_0/k^p, \\ \alpha_k &= \max \left\{ \alpha_{\min}, \frac{\nu_{k-1}\|d_{k-1}\|^2}{\Phi_k - \Phi_{\text{lev}}^k + \nu_{k-1}\|d_{k-1}\|^2} \right\}, \\ d_k &= \alpha_k g_k + (1 - \alpha_k)d_{k-1}, \\ \lambda_{k+1} &= \text{proj}_{B_\rho}(\lambda_k - \nu_k d_k).\end{aligned}$$

C Native PSD-matrix input

```
from qpsimplex.problem import SimplexQP
from qpsimplex.dual_deflected import DeflectedSubgradientSolver, DualOptions

problem = SimplexQP.from_Q(Q, q, blocks, name="my_instance")
options = DualOptions(
    max_iter=5000,
    nu0=0.5,
    step_power=0.75,
    delta0=1.0,
    delta_power=0.5,
```

```
    alpha_min=0.1,  
)  
result = DeflectedSubgradientSolver(options).solve(problem)
```

References

- [1] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge, U.K.: Cambridge University Press, 2004.
- [2] G. d’Antonio and A. Frangioni, “Convergence Analysis of Deflected Conditional Approximate Subgradient Methods,” *SIAM Journal on Optimization*, vol. 20, no. 1, pp. 357–386, 2009. doi: 10.1137/080718814.
- [3] M. Frank and P. Wolfe, “An Algorithm for Quadratic Programming,” *Naval Research Logistics Quarterly*, vol. 3, nos. 1–2, pp. 95–110, 1956. doi: 10.1002/nav.3800030109.
- [4] D. Kraft, *A Software Package for Sequential Quadratic Programming*. Technical Report DFVLR-FB 88-28, DLR German Aerospace Center, Germany, 1988.
- [5] P. Virtanen et al., “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020. doi: 10.1038/s41592-019-0686-2.